

Tribhuvan University
Institute of Science and Technology



E-governance Laboratory Report

Local Civic Problem Reporting System

B.Sc. Computer Science and Information Technology
Sixth Semester

Submitted By:-

Name: Darshan Chaulagain

Class Roll No: 07

Symbol No: 80012454

Mahendra Multiple Campus
Nepalgunj, Banke

July, 2026

CERTIFICATE

Tribhuvan University
Institute of Science and Technology
Mahendra Multiple Campus
Nepalgunj, Banke

This is to certify that the laboratory work entitled “Local Civic Problem Reporting System” submitted by Mr. Darshan Chaulagain, Class Roll No: 07, Symbol No: 80012454, of Bachelor of Science in Computer Science and Information Technology (Sixth Semester) has been carried out as a part of the requirements for the E-Governance Laboratory course under the Institute of Science and Technology, Tribhuvan University during the Academic Year 2083.

The work has been completed under my supervision and guidance. To the best of my knowledge, this laboratory work is the original work of the student and has not been submitted elsewhere for any other academic purpose.

Supervisor's Signature
Lect. Vishal Agrawal

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the Department of Computer Science and Information Technology at Mahendra Multiple Campus for providing me with the opportunity to carry out this project. I am deeply thankful to my project supervisor, Lect. Vishal Agrawal, for the valuable guidance, encouragement, and constructive feedback provided throughout the development of this project.

I would also like to extend my heartfelt thanks to my friends and classmates for their continuous support, cooperation, and helpful suggestions. Their encouragement and assistance made the project development process smoother and more enjoyable.

Finally, I am especially grateful to my family for their unconditional love, patience, and unwavering support. Their constant encouragement and belief in me motivated me to complete this project successfully.

ABSTRACT

This project presents a web-based Local Civic Problem Reporting System developed to digitize the way citizens report local infrastructure problems as an e-governance service. The system was chosen because the traditional process — visiting a ward office in person or phoning in a complaint that is logged on paper — is slow, undocumented, and gives citizens no way to track what happens to their complaint afterward. It provides a platform where citizens can register, submit a report with a photograph and an optional GPS location, and track its status in real time, while municipal staff verify, act on, and resolve reports through a role-based dashboard.

The system follows a client-server architecture and was developed using a FastAPI (Python) backend with a PostgreSQL database (hosted on Neon), and a frontend built with plain HTML, CSS and JavaScript using the Fetch API to communicate with the backend's REST endpoints. Key features include citizen registration and login, report submission with photo upload and map-based location picking (Leaflet.js / OpenStreetMap), a public feed of all reported problems with an optional anonymity toggle, a full status-tracking workflow (submitted → verified → in-progress → completed, or rejected with a reason), category-and-ward based automatic routing of reports to the responsible staff member, and an administrative layer for managing staff and citizen accounts.

To ensure security, the system implements bcrypt password hashing, stateless JWT-based authentication, role-based access control (citizen / staff / admin), server-side validation of every request through Pydantic schemas, SQLAlchemy's parameterized queries (eliminating the SQL-injection surface), strict upload validation, and an automatic logout mechanism that restricts citizens who repeatedly submit false reports. The application was deployed as two Render services — a static frontend and a FastAPI web service — backed by a managed cloud PostgreSQL database, demonstrating a functional system that improves the efficiency, transparency, and accountability of local civic problem resolution.

TABLE OF CONTENTS

Cover Page		i
CERTIFICATE		ii
ACKNOWLEDGEMENT		iii
ABSTRACT		iv
TABLE OF CONTENTS		v
LIST OF FIGURES		vi
LIST OF TABLES		vii
LIST OF ABBREVIATIONS		viii
LAB 1: Survey of E-Governance Portals		1
LAB 2: Requirement Analysis & Workflow Modeling		8
LAB 3: Backend & Database		12
LAB 4: Authentication & Access Control		17
LAB 5: Citizen Web Interface		20
LAB 6: Admin & Inter-Office Coordination		25
LAB 7: Data Protection		29
Deployment		31
Testing		33
Challenges Faced		36
Conclusion and Future Scope		37
References		38

LIST OF FIGURES

Figure 1.1 Home Page of the NIDCR Portal	2
Figure 1.2 Online Services / Notice Page of the NIDCR Portal	2
Figure 1.3 Home Page of the Department of Passports Portal	4
Figure 1.4 Online Services Page of the Passports Portal	4
Figure 1.5 Home Page of the Inland Revenue Department (IRD) Portal	6
Figure 1.6 IRD Notices and IRD NEPAL APP Services	6
Figure 2.1 As-Is Process Flowchart (Manual Complaint Process)	9
Figure 2.2 To-Be Process Flowchart (Digitized Reporting Process)	9
Figure 2.3 DFD (Context-level Diagram)	10
Figure 2.4 Use-case Diagram	11
Figure 3.1 ER Diagram — Local Civic Problem Reporting System	12
Figure 4.1 Citizen Login Interface	17
Figure 4.2 Authentication & Authorization Flow Diagram	19
Figure 5.1 Citizen Registration Interface	20
Figure 5.2 Report Submission Interface — Problem Details	21
Figure 5.3 Report Submission Interface — Photo & Map Location Picker	21
Figure 5.4 “My Reports” — Application/Status Tracking	22
Figure 5.5 Desktop View of the Public Feed	22
Figure 5.6 Mobile View of the Public Feed	23
Figure 6.1 Admin Dashboard — Create Staff Account	25
Figure 6.2 Admin Dashboard — Existing Staff Roster	26
Figure 6.3 Staff Dashboard — Reviewing an Assigned Report	27
Figure 6.4 Category + Ward Auto-Routing & Admin Override Workflow	27
Figure 8.1 Deployment Architecture of the Reporting System	31

LIST OF TABLES

Table 1.1 Comparative Analysis of E-Governance Portals	7
Table 2.1 Actors Identified	10
Table 3.1 Database Tables	12
Table 3.2 API Endpoints and Their Description	13
Table 4.1 Role-Based Access Control (RBAC) Matrix	18
Table 4.2 Sample Security Test Cases	19
Table 5.1 Integration of the Citizen Web Interface with Backend APIs	23
Table 5.2 Test Cases for the End-to-End Citizen Submission Flow	24
Table 6.1 Sample Status-Log (Audit Trail) Entries	28
Table 7.1 Security Mechanisms Implemented	29
Table 8.1 Deployment Platform Configuration	31
Table 9.1 Summary of Unit Test Cases	33
Table 9.2 Summary of System Test Cases	33
Table 9.3 End-to-End Workflow Test Results	34
Table 9.4 Security Test Results	34
Table 10.1 Challenges Faced and Resolutions	36
Table 11.1 Future Scope	37

LIST OF ABBREVIATIONS

NIDCR	National Identity Card
IRD	Inland Revenue Depart
HTML	Hypertext Markup Lang
CSS	Cascading Style Sheets
JS	JavaScript
API	Application Programmi
REST	Representational State
JWT	JSON Web Token
ORM	Object-Relational Map
SQL	Structured Query Lang
DFD	Data Flow Diagram
ER	Entity-Relationship
HTTP / HTTPS	Hypertext Transfer Prot
TLS / SSL	Transport Layer Securi
CORS	Cross-Origin Resource
XSS	Cross-Site Scripting
RBAC	Role-Based Access Cor
GPS	Global Positioning Syst
UUID	Universally Unique Ide
URL	Uniform Resource Loca

1 LAB 1: Survey of E-Governance Portals

Purpose

The purpose of this lab is to study different e-government portals of Nepal and understand how they provide digital services to citizens and businesses. This lab also helps in learning about the categories of e-government service delivery (G2C, G2B, G2E, G2G), the stages of e-government maturity, and how these portals compare against the citizen-facing e-governance service built later in this report.

Introduction

E-Governance is the use of information and communication technology (ICT) by government bodies to deliver services electronically. Instead of visiting an office for every transaction, citizens can access services through websites and mobile applications, which saves time, reduces paperwork, and improves transparency between government and the public [1]. For this lab, three official Government of Nepal portals were surveyed and classified by e-government category, maturity stage, and service-delivery model.

Government Portal 1: Department of National ID and Civil Registration (NIDCR)

Website: <https://donidcr.gov.np>

The Department of National ID and Civil Registration, under the Ministry of Home Affairs, issues the National Identity Card and manages civil registration records (birth, death, marriage, migration) for citizens of Nepal. Its portal is the main access point for checking National ID status, applying for civil-registration events online, and reaching related social-security services.

Main Services

- National Identity Card enrolment and status
- Online civil-event registration (birth, death, marriage, migration)
- National ID Search
- Social security registration

E-Government Category

- Government to Citizen (G2C) — services are aimed directly at individual citizens.

Stage of E-Government

- Transaction Stage — citizens can submit civil-registration requests and check application status online instead of visiting an office for every step.

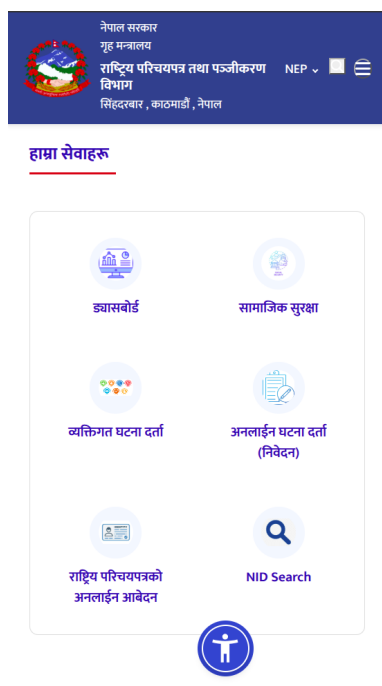


Figure 1.1 Home Page of the NIDCR Portal



Figure 1.2 Online Services / Notice Page of the NIDCR Portal

Observation: The NIDCR portal centralizes identity and civil-registration services that previously required a physical visit to a District Administration Office or ward office, shortening the path from application to a trackable status.

Government Portal 2: Department of Passports

Website: <https://nepalpassport.gov.np>

The Department of Passports, established in 2068 B.S., provides online passport services for Nepali citizens. Through its website, applicants can register an account, complete the online passport application form, book an appointment slot, and track the status of an existing application [3].

Main Services

- Online passport application
- Appointment booking
- Passport status tracking
- General passport information / FAQs

E-Government Category

- Government to Citizen (G2C).

Stage of E-Government

- Transaction Stage — citizens complete the bulk of the application before visiting the passport office only for biometric capture.



Figure 1.3 Home Page of the Department of Passports Portal

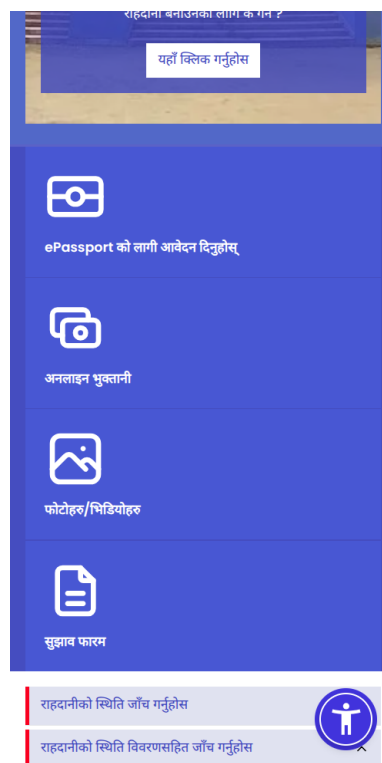


Figure 1.4 Online Services Page of the Passports Portal

Government Portal 3: Inland Revenue Department (IRD)

Website: <https://ird.gov.np>

The Inland Revenue Department is responsible for tax administration in Nepal. Through its portal and the companion “IRD Nepal” app, taxpayers can register for a Permanent Account Number (PAN), file VAT returns, search tax-clearance status, and access tax calculators without visiting a tax office for routine work [4].

Main Services

- PAN registration
- Online VAT filing
- Tax clearance search
- Tax rate & incentive information

E-Government Category

- Government to Business (G2B) — the portal mainly serves taxpayers and businesses.

Stage of E-Government

- Transaction Stage — registration, filing and clearance checks are completed online.

Comparative Analysis

Table 1.1 Comparative Analysis of E-Governance Portals

S.N	Portal	Category	Stage	Main Users	Main Purpose
1	NIDCR	G2C	Transaction	Citizens	National ID and civil-registration services
2	Department of Passports	G2C	Transaction	Citizens	Passport application and tracking
3	Inland Revenue Department	G2B	Transaction	Businesses / Taxpayers	Online tax services

Findings

All three portals digitize a previously office-bound, paper-based service and have reached the Transaction stage of e-government maturity: citizens and businesses can complete a meaningful part of a government process online rather than only reading information about it. The same Transaction-stage pattern — register, submit a structured request with supporting evidence, and track a status — is the pattern the Local Civic Problem Reporting System (this project) applies to municipal infrastructure complaints in Lab 2 onward.

Conclusion

This survey shows that Nepal's e-governance portals are converging on the same underlying workflow — identity-backed registration, structured digital submission, and real-time status tracking — regardless of the specific service. Studying these three portals directly informed the requirement analysis in Lab 2, where the same pattern is applied to the manual civic-complaint process.

2 LAB 2: Requirement Analysis & Workflow Modeling

Purpose

The purpose of this lab is to analyze the existing manual process for reporting a local infrastructure problem, identify the actors and their responsibilities, and model the proposed digital system using process flowcharts, a context-level Data Flow Diagram (DFD), and a UML use-case diagram.

Introduction of the Chosen Manual Government Service

The chosen manual government service is local infrastructure-problem reporting — potholes, broken water pipes, non-functioning streetlights, damaged public property, and similar issues. In the traditional process, a citizen must travel to the ward office (or phone in) to lodge a complaint, which is written into a paper register if a staff member happens to be available, then relayed verbally or by phone to the relevant department. There is no receipt, no reference number, and no way for the citizen to later check whether anyone acted on it.

The proposed Local Civic Problem Reporting System digitizes this process end to end: citizens register and submit a report with a category, ward number, description, photo and optional GPS pin; the system automatically routes it to the staff member responsible for that category and ward; every status change is written to an immutable log; and the citizen can track progress and message staff directly — all without a repeat visit to an office.

Process Flowcharts

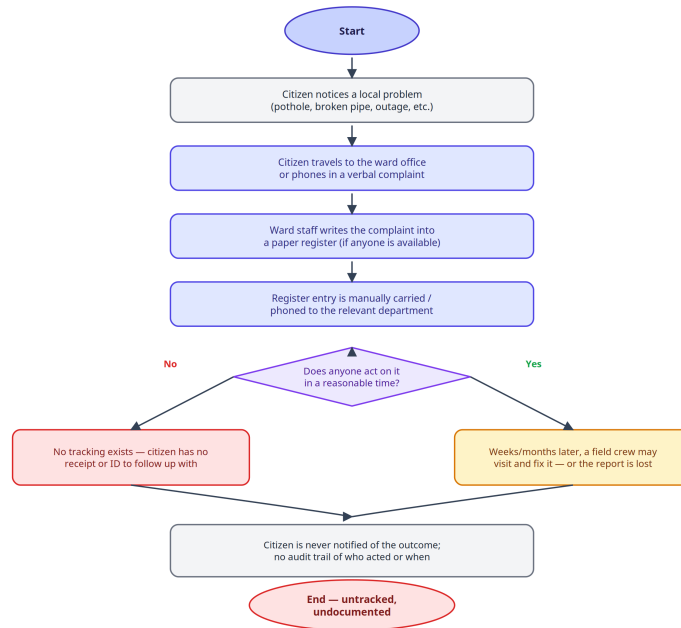


Figure 2.1 As-Is Process Flowchart (Manual Complaint Process)

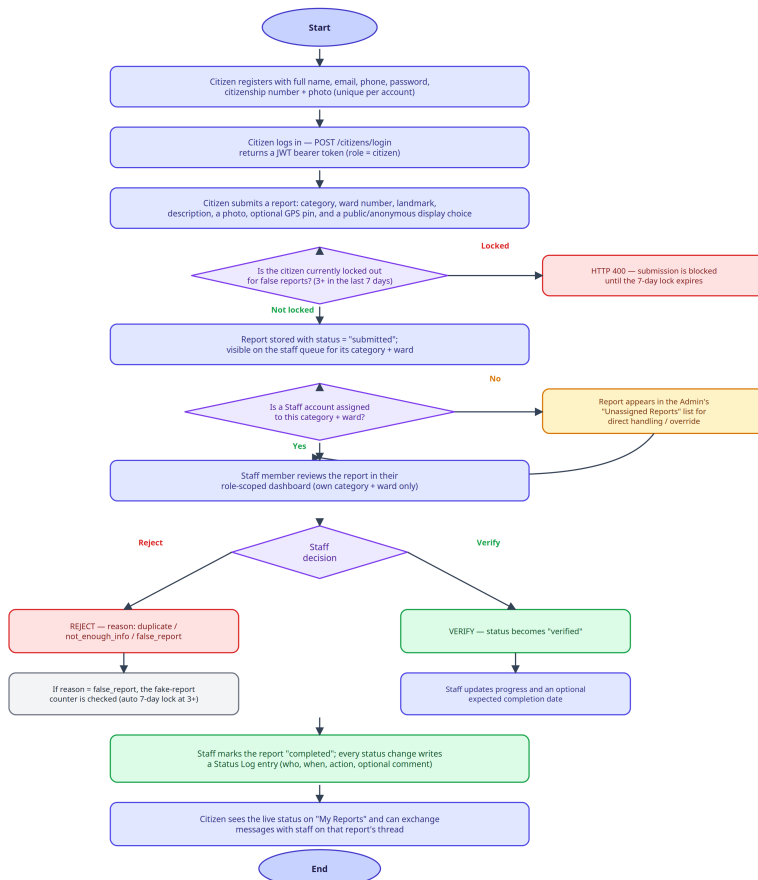


Figure 2.2 To-Be Process Flowchart (Digitized Reporting Process)

Actors Identified

Table 2.1 Actors Identified

S.N	Actor	Role in the System
1	Citizen	Registers with a citizenship number and photo, logs in, submits and edits/deletes an unreviewed report (photo + optional GPS), tracks status on “My Reports”, browses the public feed, and messages staff on a report's thread.
2	Staff (Department)	A category- and ward-scoped account created by an Admin. Reviews only the reports that match their own category and ward; verifies, rejects (with a reason), progresses, or completes them; and messages the reporting citizen.
3	System Admin	Creates/disables/enables/deletes Staff and Citizen accounts; has full visibility across every report; can directly handle or override reports that have no Staff assigned to their category + ward (“Unassigned Reports”) or any report at all.

Context-level DFD and Use-case Diagram

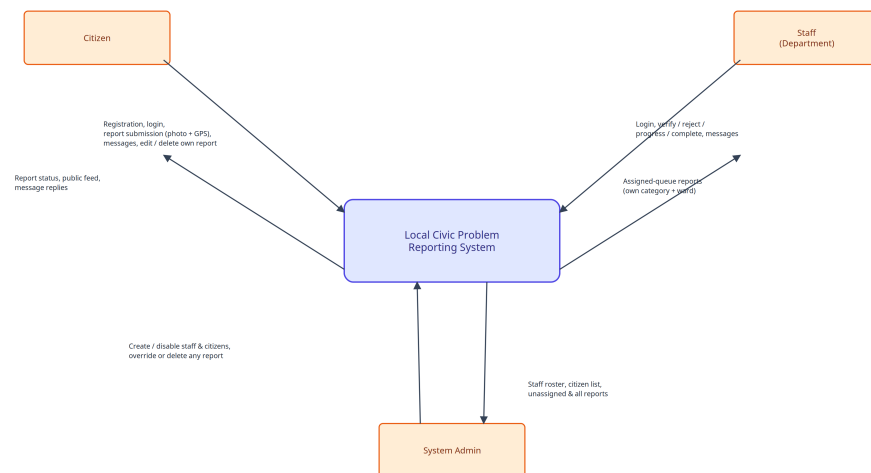


Figure 2.3 DFD (Context-level Diagram)

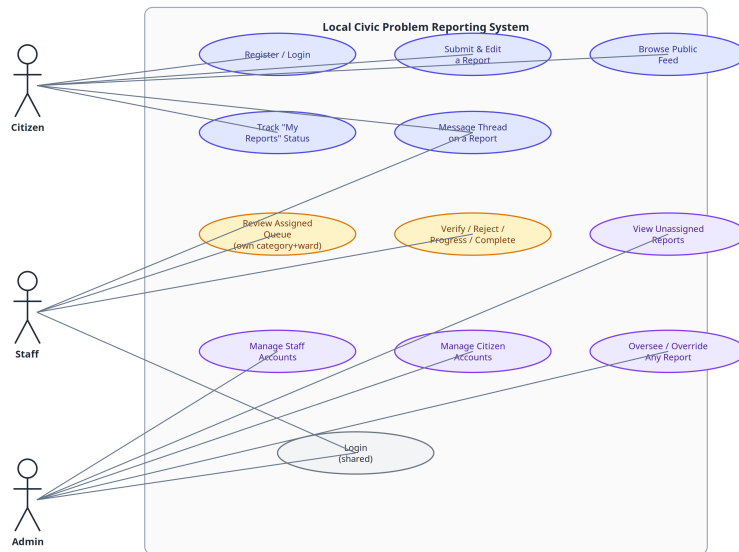


Figure 2.4 Use-case Diagram

3 LAB 3: Backend & Database

Purpose

The purpose of this lab is to document the backend and database implementation of the Local Civic Problem Reporting System: the relational schema, the REST API surface built with FastAPI, and the security measures applied at the data layer.

Relational Schema / ER Diagram

The database is PostgreSQL, accessed through the SQLAlchemy ORM, and stores five tables: citizens, staff, reports, status_logs and messages. Primary keys are UUIDs; foreign keys tie every report back to its citizen, every status-log row to the report and the staff member who made the change, and every message to its report and its sender.

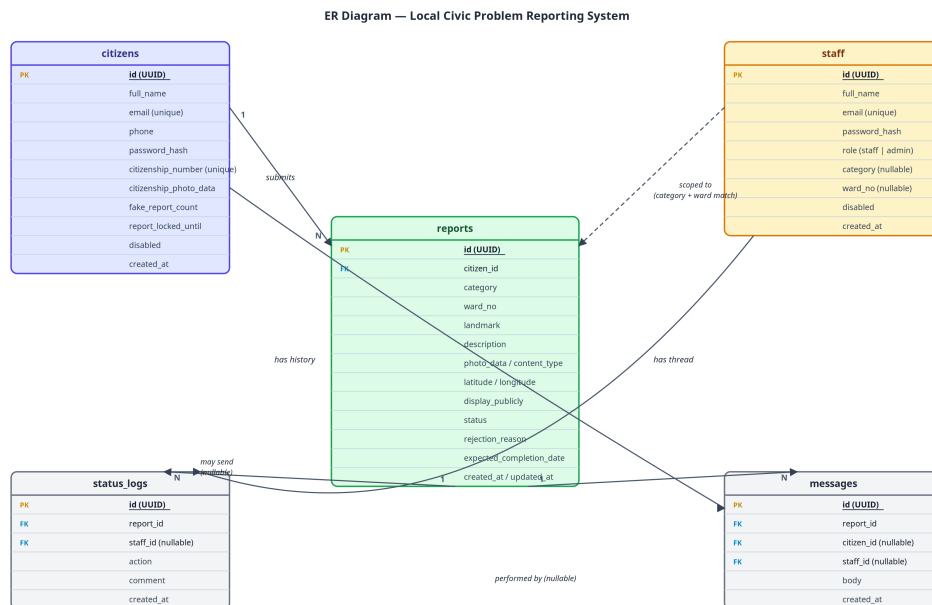


Figure 3.1 ER Diagram — Local Civic Problem Reporting System

Table 3.1 Database Tables

Table	Purpose	Key Columns
citizens	Registered citizen accounts	id (PK), email (unique), citizenship_number (unique), password_hash, fake_report_count, report_locked_until, disabled
staff	Department Staff and Admin accounts	id (PK), email (unique), password_hash, role (staff admin), category, ward_no, disabled

Table	Purpose	Key Columns
reports	One submitted civic problem	id (PK), citizen_id (FK), category, ward_no, photo_data, status, rejection_reason, expected_completion_date
status_logs	Immutable audit trail of every status change	id (PK), report_id (FK), staff_id (FK, nullable), action, comment, created_at
messages	Citizen ↔ Staff conversation on a report	id (PK), report_id (FK), citizen_id / staff_id (FK, nullable — exactly one is set), body, created_at

API Endpoints Implemented

Table 3.2 API Endpoints and Their Description

Router	Endpoint	Method	Description
citizens	/citizens/register	POST	Registers a citizen (hashes the password, stores citizenship number + photo, rejects duplicate email/citizenship number).
citizens	/citizens/login	POST	Verifies credentials and returns a JWT (role = citizen).
citizens	/citizens/me	GET / PATCH	Reads or updates the logged-in citizen's own profile.
reports	/reports	POST	Submits a new report (category, ward, landmark, description, photo, optional GPS); blocked if the citizen is currently locked out.
reports	/reports	GET	Public feed of reports, filterable by category / status.
reports	/reports/mine	GET	The logged-in citizen's own reports.
reports	/reports/{id}	GET	Full detail incl. status history; reveals the real reporter name only to staff or the report's own citizen.
reports	/reports/{id}	PATCH / DELETE	Citizen edits/deletes their own report — only while it is still “submitted”.
staff	/staff/login	POST	Verifies credentials and returns a JWT (role = staff admin).
staff	/staff/reports	GET	The logged-in Staff member's own category + ward queue.
staff	/staff/reports/{id}/verify	PATCH	Marks a report verified; writes a status_log row.

Router	Endpoint	Method	Description
staff	/staff/reports/{id}/reject	PATCH	Rejects with a reason; feeds the fake-report lockout rule.
staff	/staff/reports/{id}/progress	PATCH	Marks in_progress with an expected completion date.
staff	/staff/reports/{id}/complete	PATCH	Marks a report completed.
admin	/admin/staff	POST / GET	Creates a Staff/Admin account or lists all of them.
admin	/admin/staff/{id}/disable enable	PATCH	Suspends or restores a Staff account.
admin	/admin/citizens	GET	Lists every registered citizen.
admin	/admin/reports/unassigned	GET	Reports whose category + ward has no Staff yet.
admin	/admin/reports	GET / PATCH / DELETE	System-wide oversight and override of any report.
messages	/reports/{id}/messages	GET / POST	Reads or posts to the private thread between the reporting citizen and the assigned Staff member.

API Documentation — Worked Examples

1. Register API

Method: POST **Endpoint:** /citizens/register (multipart/form-data)

Registers a new citizen. The password is hashed with bcrypt before storage; the citizenship number is validated against a pattern and checked for uniqueness before the account is created.

```
POST /citizens/register (multipart/form-data)
  full_name=Ram Bahadur
  email=ram@example.com
  phone=9800000000
  password=*****
  citizenship_number=12-34-56-78901
  citizenship_photo=<file>

201 Created
{
  "id": "b6e2...c9",
  "full_name": "Ram Bahadur",
  "email": "ram@example.com",
  "disabled": false
}
```

2. Submit Report API

Method: POST **Endpoint:** /reports (multipart/form-data, Bearer token required)

Records a citizen's report. The server re-checks the citizen's lockout status before accepting the submission and rejects the request with HTTP 400 if the 7-day false-report lock is active.

```
POST /reports (Authorization: Bearer <citizen token>)
  category=road
  ward_no=6
  landmark=Near Buspark
  description=Road washed out after rain
  photo=<file>
  latitude=28.05 longitude=81.61

201 Created
{
  "id": "f1a0...3e",
  "status": "submitted",
  "reporter_name": "Ram Bahadur"
}
```

3. Status / Detail Retrieval API

Method: GET **Endpoint:** /reports/{id}

Returns the full detail of one report, including its status-log history. The reporter's real name is only revealed to Staff or to the citizen who filed it — every other viewer sees “Anonymous Citizen” when the citizen chose not to display their name publicly.

```
GET /reports/fla0...3e

200 OK
{
  "id": "fla0...3e",
  "category": "road", "ward_no": 6,
  "status": "in_progress",
  "expected_completion_date": "2026-12-11",
  "status_logs": [
    {"action": "verified", "created_at": "2026-09-13T11:40:02Z"},
    {"action": "in_progress", "created_at": "2026-09-14T09:12:44Z"}
  ]
}
```

Security Measures

- Passwords are hashed with bcrypt (passlib.CryptContext) — never stored or compared in plain text.
- Citizenship number, email uniqueness, and file type/size are validated server-side with Pydantic / FastAPI form validators, independent of any client-side check.
- All database access goes through SQLAlchemy's ORM query builder, which parameterizes every value and eliminates the SQL-injection surface — no raw string-built SQL is used anywhere in the routers.
- Report photos and citizenship photos are re-processed on upload (type/size enforced) and stored as BYTEA rows in Postgres rather than on local disk, so they are never served from an untrusted, user-controlled file path.
- CORS is restricted to an explicit allow-list of origins read from configuration (CORS_ORIGINS), not a wildcard.

Version Control

The project is tracked with Git and hosted on GitHub at github.com/MrLeo0087/Local-reporting-system-, with the backend (FastAPI app) and frontend (static HTML/CSS/JS) as sibling top-level folders in the same repository. Commits were made incrementally per feature — database models, each router, authentication, and later bug fixes such as correcting message-thread attribution and refreshing the citizen identity shown to staff.

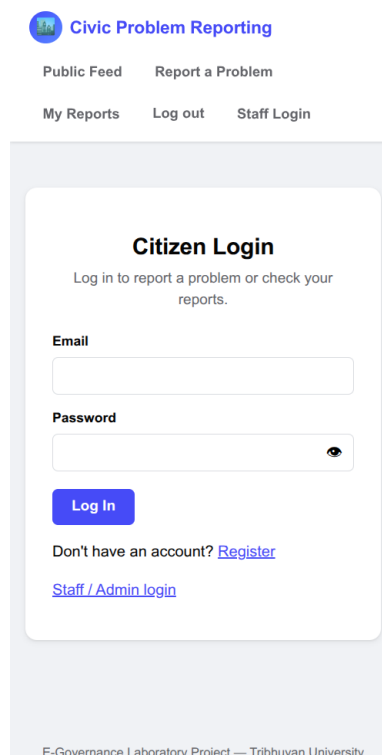
4 LAB 4: Authentication & Access Control

Purpose

The purpose of this lab is to document the authentication and access-control mechanism of the system: how a user's identity is verified, how a session is represented statelessly with a token, and how each role's permissions are enforced on every request.

Password Policy and Login

- Passwords are hashed with bcrypt before being stored; the plain password is never persisted.
- A citizen logs in with email + password against the citizens table; a Staff or Admin logs in with email + password against the staff table — two separate login endpoints issuing two separate token types.
- A disabled account (disabled = true) is rejected at login with HTTP 403, even with a correct password.
- Login failures return a generic “Incorrect email or password” message, which avoids revealing whether the email exists in the system.



The screenshot shows the 'Civic Problem Reporting' web application. At the top, there is a navigation bar with a logo and the title 'Civic Problem Reporting'. Below the title, there are links for 'Public Feed', 'Report a Problem', 'My Reports', 'Log out', and 'Staff Login'. The main content area is a 'Citizen Login' form. The form has a title 'Citizen Login' and a subtitle 'Log in to report a problem or check your reports.' It contains two input fields: 'Email' and 'Password'. The 'Password' field has a toggle icon for visibility. Below the input fields is a blue 'Log In' button. At the bottom of the form, there are links for 'Don't have an account? Register' and 'Staff / Admin login'. The footer of the page reads 'E-Governance Laboratory Project — Tribhuvan University'.

Figure 4.1 Citizen Login Interface

Role-Based Access Control (RBAC)

Table 4.1 Role-Based Access Control (RBAC) Matrix

S.N	Feature / Module	Citizen	Staff	Admin
1	Register account	✓	✗ (created by Admin)	✗
2	Login	✓	✓	✓
3	Browse public feed	✓	✓	✓
4	Submit / edit own report	✓	✗	✗
5	Review assigned queue (verify/reject/progress/complete)	✗	✓ (own category+ward only)	✗
6	Message thread on a report	✓ (own report)	✓ (own category+ward)	✗
7	Manage Staff accounts	✗	✗	✓
8	Manage Citizen accounts	✗	✗	✓
9	View / override any report	✗	✗	✓
10	View unassigned reports	✗	✗	✓

Session Handling

The system is stateless: there is no server-side session store. Each login issues a signed JWT (HS256) containing the user's id, role and — for Staff — category and ward number, valid for 120 minutes (JWT_EXPIRE_MINUTES). The frontend keeps the token in localStorage (citizen_token / staff_token, kept separate so one role's token can never be reused as the other) and attaches it as an Authorization: Bearer header on every request. Logging out simply clears the stored token and role; because the token itself is never recorded server-side, an expired or cleared token cannot be replayed.

Authentication Flow Diagram

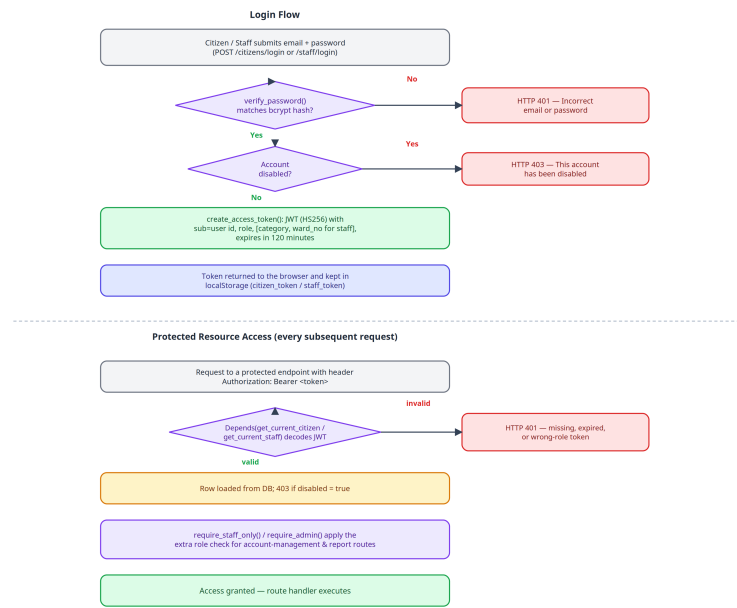


Figure 4.2 Authentication & Authorization Flow Diagram

Sample Security Test Cases

Table 4.2 Sample Security Test Cases

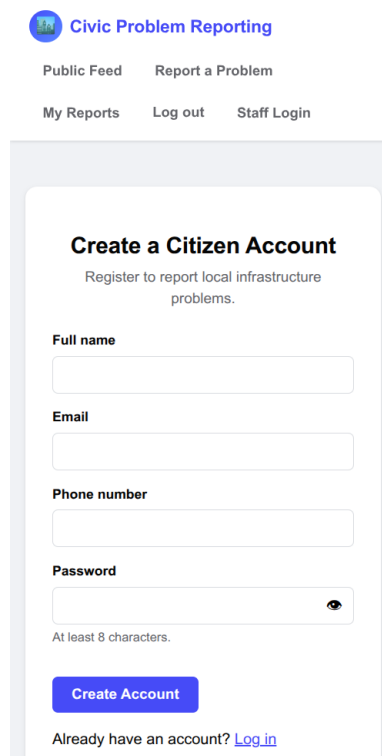
S.N	Test Case	Expected Result	Status
1	Valid citizen login	JWT issued, redirected to citizen dashboard	Pass
2	Login with wrong password	HTTP 401, generic error message	Pass
3	Login to a disabled account	HTTP 403, “account disabled”	Pass
4	Staff token used on an admin-only route	HTTP 403 via require_admin()	Pass
5	Citizen token used on a staff route	HTTP 401 — role in token $\neq$ staff/admin	Pass
6	Expired / tampered JWT on a protected route	HTTP 401, access denied	Pass
7	Staff acting on a report outside their category/ward	HTTP 403 / 404 (not in their queue)	Pass
8	Logout	Token cleared from localStorage; protected pages redirect to login	Pass

5 LAB 5: Citizen Web Interface

Purpose

The purpose of this lab is to document the citizen-facing frontend: registration, login, report submission, status tracking, and the public feed, all integrated against the Lab 3 backend APIs.

UI Screenshots



The screenshot displays the 'Civic Problem Reporting' web application. At the top, there is a navigation bar with a logo and the title 'Civic Problem Reporting'. Below the title, there are links for 'Public Feed', 'Report a Problem', 'My Reports', 'Log out', and 'Staff Login'. The main content area is titled 'Create a Citizen Account' with a subtitle 'Register to report local infrastructure problems.' Below this, there are four input fields: 'Full name', 'Email', 'Phone number', and 'Password'. The 'Password' field has a toggle icon for visibility and a note 'At least 8 characters.' Below the input fields is a blue 'Create Account' button. At the bottom, there is a link 'Already have an account? Log in'.

Figure 5.1 Citizen Registration Interface

Description: Citizens register with their full name, email, phone, a password, and a citizenship number plus a photo of the citizenship certificate, which the server checks for uniqueness before creating the account.

Civic Problem Reporting

Public Feed **Report a Problem** My Reports

Log out Staff Login

Report a Problem

Tell us what's wrong and where — a photo helps staff verify it faster.

Category
Road

Ward number

Nearby landmark
e.g. Near City Hospital gate

Description
Describe the problem in detail

Photo (JPG or PNG, max 5MB)
Choose File No file chosen

Pin the location on the map (optional)

Figure 5.2 Report Submission Interface — Problem Details

Civic Problem Reporting

Public Feed **Report a Problem** My Reports

Log out Staff Login

Photo (JPG or PNG, max 5MB)
Choose File No file chosen

Pin the location on the map (optional)

No location pinned yet — click the map to add one.

☒ Show my name publicly on this report (otherwise it shows as "Anonymous Citizen")

Submit Report

Figure 5.3 Report Submission Interface — Photo & Map Location Picker

Description: A logged-in citizen selects a category and ward, describes the problem, attaches a required photo, and may drop a pin on a Leaflet/OpenStreetMap map for the exact location. A checkbox controls whether the citizen's name is shown publicly on the feed or masked as “Anonymous Citizen”.

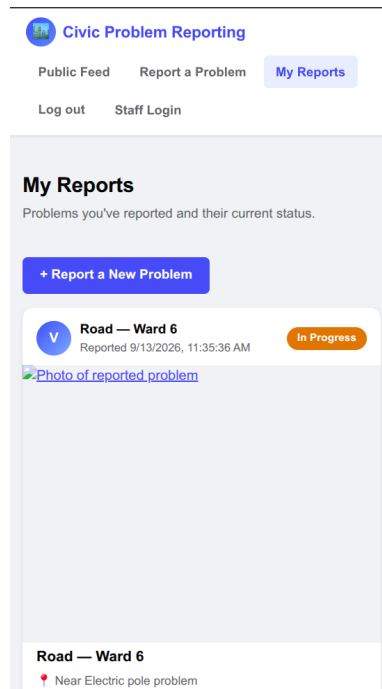


Figure 5.4 “My Reports” — Application/Status Tracking

Description: Citizens see every report they have filed together with its live status badge (Submitted, In Progress, Rejected, Completed) without needing to contact the ward office.

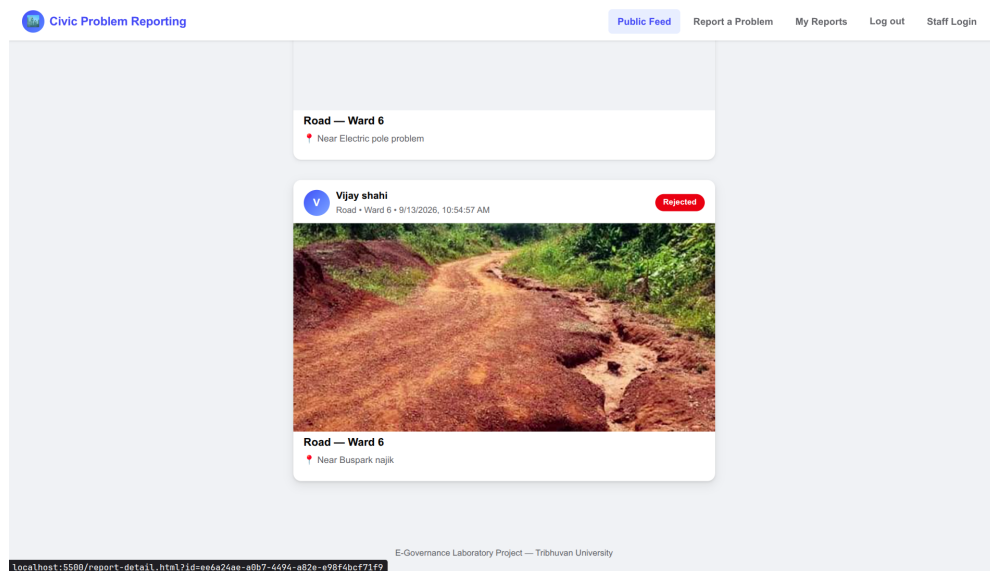


Figure 5.5 Desktop View of the Public Feed

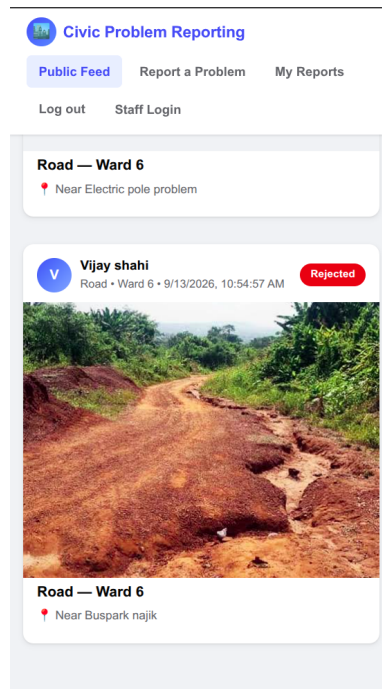


Figure 5.6 Mobile View of the Public Feed

Description: The public feed lists every report system-wide (filterable by category and status) so any citizen can see what has already been reported nearby before filing a duplicate.

Form Validation Approach

- Client-side: required-field, email-format, minimum-password-length and file-type/size checks give the citizen immediate feedback before a request is even sent.
- Server-side (authoritative): FastAPI/Pydantic re-validate every field regardless of what the browser already checked — citizenship number against a strict pattern, email format, password length ≥ 8 , and photo re-processing (type + ≤ 5 MB) via Pillow.
- Duplicate-account and duplicate-citizenship-number checks happen against the database at registration time, not just in the form.

Integration with Backend APIs

Table 5.1 Integration of the Citizen Web Interface with Backend APIs

Interface	API Used	Purpose
register.html	POST /citizens/register	Creates a new citizen account
login.html	POST /citizens/login	Authenticates and stores the JWT

Interface	API Used	Purpose
submit-report.html	POST /reports	Submits a new report with photo + optional GPS
my-reports.html	GET /reports/mine	Lists the citizen's own reports and status
feed.html	GET /reports	Public feed of all reports
report-detail.html	GET/POST /reports/{id}/messages	Report detail and the citizen ↔ staff message thread

Responsive / Mobile-Friendly Design

The frontend uses plain CSS with flexible widths, a mobile-first layout, and a viewport meta tag, so the same HTML/CSS serves both breakpoints without a separate mobile site. Figures 5.5 and 5.6 above show the same Public Feed page rendered at desktop and mobile widths.

Test Cases — End-to-End Citizen Submission Flow

Table 5.2 Test Cases for the End-to-End Citizen Submission Flow

S.N	Test Scenario	Input	Expected Result	Status
1	Register with valid details	Valid citizen info + photo	Account created, 201	Pass
2	Register with a duplicate citizenship number	Already-used number	400 — duplicate rejected	Pass
3	Login with registered account	Correct email/password	JWT issued	Pass
4	Submit a report	Valid category/ward/ photo	201, status = submitted	Pass
5	Edit a report after Staff has verified it	PATCH on a verified report	400 — no longer editable	Pass
6	3rd false_report rejection within 7 days	Staff rejects with reason=false_report	Citizen locked out 7 days	Pass
7	View own report status	GET /reports/mine (with token)	Live status returned	Pass

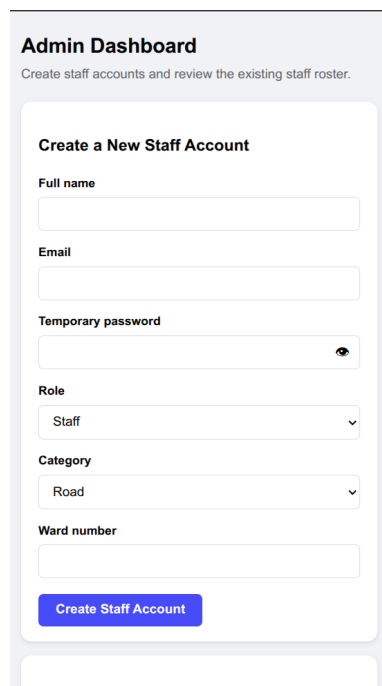
6 LAB 6: Admin & Inter-Office Coordination

Purpose

The purpose of this lab is to document how Staff (department) accounts and the System Admin coordinate to move a report from submission to resolution, and how every action is recorded for accountability.

System Admin

The System Admin manages the platform: creating, disabling, enabling and deleting Staff and Citizen accounts, and holding full oversight of every report in the system. Unlike a two-tier ward/district approval chain, this system uses a single review tier — a report is routed straight to whichever Staff account is scoped to its category and ward. The Admin does not sit in that review path day to day; it steps in only when a category + ward has no Staff assigned yet (“Unassigned Reports”) or when an override is needed.



The image shows a mobile app interface for the Admin Dashboard. At the top, the title 'Admin Dashboard' is followed by the subtitle 'Create staff accounts and review the existing staff roster.' Below this is a section titled 'Create a New Staff Account'. The form contains several input fields: 'Full name', 'Email', and 'Temporary password' (which has an eye icon for toggling visibility). There are also two dropdown menus for 'Role' (currently set to 'Staff') and 'Category' (currently set to 'Road'). A 'Ward number' input field is at the bottom of the form. A blue button labeled 'Create Staff Account' is positioned below the 'Ward number' field. The entire form is enclosed in a light gray border.

Figure 6.1 Admin Dashboard — Create Staff Account

Category

Road

Ward number

Create Staff Account

Existing Staff Accounts

NAME	EMAIL	ROLE	CAT
Prakash	prakashbudha0087@gmail.com	staff	Elec Line
Nisan	nisanshahi0087@gmail.com	staff	Roa
Darshan	iamleo0087@gmail.com	admin	—

E-Governance Laboratory Project — Tribhuvan University

Figure 6.2 Admin Dashboard — Existing Staff Roster

Staff Review Queue

A Staff account is created by the Admin with a fixed category (e.g. road, electric, water_supply, public_property, other) and ward number. Logging in returns a token carrying that scope, and every subsequent query (GET /staff/reports and the verify/reject/progress/complete actions) is filtered to reports matching that exact category and ward — a Staff account can never see or act on a report outside its own assignment.

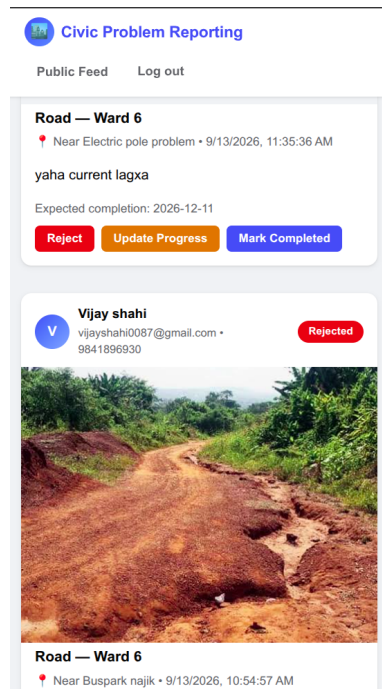


Figure 6.3 Staff Dashboard — Reviewing an Assigned Report

Category + Ward Routing and Admin Override

In place of a ward-office-to-district-office handoff, this system performs automatic routing: every incoming report is matched to a Staff account by category and ward at read time. When no such Staff account exists yet, the report surfaces in the Admin's “Unassigned Reports” view for direct handling. The Admin can, at any time and regardless of assignment, view every report system-wide and override its status or delete it — the accountability safeguard that replaces a formal escalation chain.

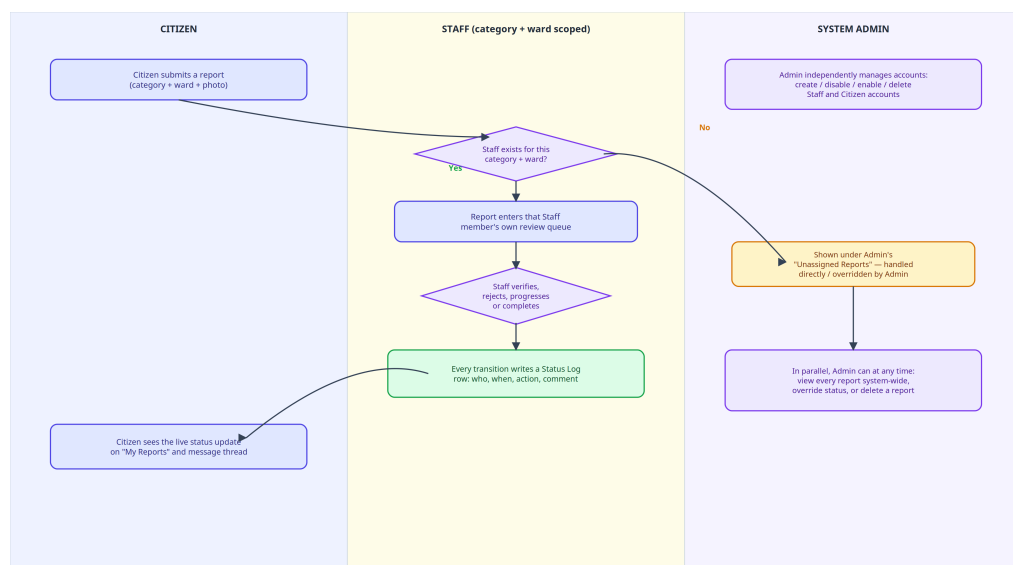


Figure 6.4 Category + Ward Auto-Routing & Admin Override Workflow

Sample Status-Log (Audit Trail) Entries

Every verify / reject / progress / complete action writes one row to `status_logs`, capturing who acted, when, and an optional comment — this is what powers the “status history” timeline shown on a report's detail page.

Table 6.1 Sample Status-Log (Audit Trail) Entries

Timestamp	Staff	Action	Comment
2026-09-13 11:35 AM	— (citizen)	submitted	Report filed by citizen
2026-09-13 11:40 AM	Prakash (road, Ward 6)	verified	Confirmed from photo
2026-09-14 09:12 AM	Prakash (road, Ward 6)	in_progress	Crew scheduled, ETA 2026-12-11
2026-09-20 10:54 AM	Nisan (road, Ward 6)	rejected	Duplicate of an earlier report

7 LAB 7: Data Protection

Purpose

The purpose of this lab is to document how sensitive data is protected in transit and at rest, and the backup/recovery and privacy practices adopted by the system.

HTTPS / TLS

During development the API was exercised over plain HTTP on localhost. In deployment, both Render services — the static frontend and the FastAPI backend — are served over HTTPS by Render's own TLS termination, so every request between the browser and both services is encrypted in transit.

Sensitive Data Protection

Sensitive data handled by the system includes citizen names, email addresses, phone numbers, citizenship numbers, and photographs (both citizenship documents and report evidence). Passwords are hashed with bcrypt; citizenship-photo and report-photo binary data is stored inside PostgreSQL (BYTEA) rather than on a shared filesystem; the message thread on a report is only ever visible to the reporting citizen and the assigned Staff member; and citizens can view only their own account and their own reports.

Security Mechanisms Implemented

Table 7.1 Security Mechanisms Implemented

Mechanism	Purpose
bcrypt password hashing (passlib)	Passwords are never stored or compared in plain text
JWT bearer tokens (python-jose, HS256)	Stateless, tamper-evident session identity, 120-minute expiry
Role-scoped FastAPI dependencies	get_current_citizen / require_staff_only / require_admin enforce RBAC on every route
SQLAlchemy ORM (parameterized queries)	Eliminates the SQL-injection surface
Pydantic schema validation	Rejects malformed input before it reaches business logic
Explicit CORS allow-list	Only configured frontend origins may call the API
BYTEA photo storage in Postgres	

Mechanism	Purpose
	Photos persist across restarts and are never served from an unauthenticated file path
Fake-report lockout	Automatically restricts a citizen after 3 false_report rejections within 7 days
Anonymity-aware serialization	“Anonymous Citizen” shown publicly unless the viewer is Staff or the report's own citizen

Backup and Recovery Procedure

Backup

- Take a logical dump of the Neon PostgreSQL database with `pg_dump` (or Neon's own branch/snapshot feature) and store it with a dated filename, e.g. `backup_2026-09-20.sql`.
- Because report and citizenship photos are stored as BYTEA columns inside the same database (not on disk), a full `pg_dump` already captures them — there is no separate uploads folder to back up, unlike a filesystem-based design.
- Store the `JWT_SECRET_KEY` and `DATABASE_URL` values (from the deployment environment) in a separate, access-controlled secrets record — never inside the same backup archive as the data it protects.

Recovery

- Provision a fresh PostgreSQL database (or a new Neon branch).
- Restore with `pg_restore / psql < backup_2026-09-20.sql`.
- Point `DATABASE_URL` at the restored database and redeploy the backend service.
- Log in as an Admin and spot-check a few citizen and report records to confirm the restore is complete before resuming normal operation.

Security and Privacy Policy

Only authenticated users may access protected resources; citizens can access only their own account and reports; Staff can act only on reports inside their assigned category and ward; every account-management and report-override action taken by an Admin is available for review through the status-log audit trail; and personal data collected at registration is used only for identity verification and report handling — it is not shared with any party outside the system.

8 Deployment

Purpose

This section documents how the completed system is deployed and made accessible over the internet.

Deployment Architecture

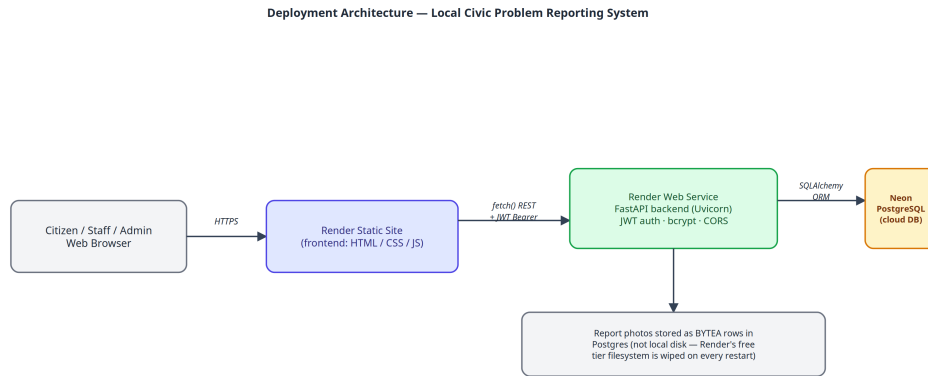


Figure 8.1 Deployment Architecture of the Reporting System

Platform Used

Table 8.1 Deployment Platform Configuration

Component	Description
Frontend hosting	Render Static Site — serves the plain HTML/CSS/JS frontend directly
Backend hosting	Render Web Service — runs the FastAPI app under Uvicorn (local-reporting-system.onrender.com)
Database	Managed PostgreSQL on Neon, connected via DATABASE_URL
Backend start command	uvicorn app.main:app --host 0.0.0.0 --port \$PORT
CORS	CORS_ORIGINS environment variable lists the exact frontend origin(s) allowed to call the API
Secrets	DATABASE_URL and JWT_SECRET_KEY set as Render environment variables, never committed to Git

README.md and .env.example

The repository's top-level README.md documents the project layout, local setup for both the FastAPI backend and the static frontend, and the two-service Render deployment steps. backend/.env.example lists every configuration variable the backend needs (DATABASE_URL, JWT_SECRET_KEY, JWT_ALGORITHM, JWT_EXPIRE_MINUTES, UPLOAD_DIR, MAX_UPLOAD_MB, CORS_ORIGINS, PORT) with safe placeholder values, so a real .env is never committed to version control.

9 Testing

Purpose

Testing verifies that each module behaves correctly in isolation, that the full citizen-to-resolution workflow works end to end, and that the security controls from Lab 4 and Lab 7 hold up under adversarial input.

Unit Tests

Table 9.1 Summary of Unit Test Cases

Test ID	Module	Description	Expected Result	Status
UT-01	Registration	Register with valid data	Citizen account created	Pass
UT-02	Registration	Register with a duplicate citizenship number	400, duplicate rejected	Pass
UT-03	Login	Login with valid credentials	JWT returned	Pass
UT-04	Login	Login with invalid password	401 returned	Pass
UT-05	Report submission	Submit a report with a photo	Report stored, status = submitted	Pass
UT-06	Fake-report lock	3rd false_report rejection within 7 days	report_locked_until set	Pass

System Tests

Table 9.2 Summary of System Test Cases

Test ID	Feature	Expected Result	Status
ST-01	Citizen registration & login	Account created, JWT-based session established	Pass
ST-02	Report submission & routing	Report visible only to the matching category+ward Staff	Pass
ST-03	Staff review actions	Verify/reject/progress/complete each write one status_log row	Pass
ST-04	Admin account management	Admin can create/disable/enable Staff and Citizen accounts	Pass

Test ID	Feature	Expected Result	Status
ST-05	Message thread	Only the reporting citizen and assigned Staff can read/post	Pass

End-to-End Workflow Test Results

Table 9.3 End-to-End Workflow Test Results

Step	Action	Expected Result	Status
1	Citizen registers and logs in	Account created; JWT issued	Pass
2	Citizen submits a report (category=road, ward=6)	Report stored, status=submitted	Pass
3	Matching Staff logs in and opens their queue	Report appears in the Staff's own list	Pass
4	Staff verifies the report	Status → verified, status_log written	Pass
5	Staff sets progress with an expected completion date	Status → in_progress	Pass
6	Staff marks the report completed	Status → completed, status_log written	Pass
7	Citizen checks “My Reports”	Live “completed” status displayed	Pass

Security Test Results

Table 9.4 Security Test Results

S.N	Security Test	Expected Result	Status
1	SQL injection via report/citizen text fields	Rejected / neutralized by parameterized ORM queries	Pass
2	Cross-role token reuse (citizen token on a staff route)	401 — role check fails	Pass
3	Staff acting outside assigned category/ward	403 / 404 — blocked by scope check	Pass
4	Password stored in the database	Stored only as a bcrypt hash	Pass
5	Disabled account login	403 — login blocked	Pass
6	Direct API access without a token	401 on every protected endpoint	Pass

S.N	Security Test	Expected Result	Status
7	Duplicate voting-equivalent (editing a reviewed report)	400 — no longer editable once reviewed	Pass

10 Challenges Faced

Table 10.1 Challenges Faced and Resolutions

S.N	Challenge	Resolution
1	Render's free-tier web service wipes its local disk on every restart, so photos saved to a local uploads/ folder disappeared.	Switched report and citizenship photos to BYTEA columns inside Postgres, served through a dedicated /reports/{id}/photo endpoint instead of a static file path.
2	Deciding how to route a report to the right department without a manual dispatcher.	Modeled category + ward as attributes on both Staff and Report, and filtered the Staff queue query by an exact match on both.
3	Preventing the public anonymity option from being abused to file bad-faith reports consequence-free.	Anonymity only hides the reporter's name on the public feed; Staff and the reporting citizen always see the real identity, and a fake-report lockout still applies regardless of the anonymity setting.
4	Keeping a citizen token and a staff token from ever being interchangeable.	Issued the role directly inside the JWT payload and checked it in two separate FastAPI dependencies (get_current_citizen vs. get_current_staff).
5	CORS errors once the frontend and backend were split across two separate Render services.	Added the deployed frontend's exact origin to the backend's CORS_ORIGINS environment variable rather than allowing all origins.
6	A stale citizen identity was briefly attributed to staff replies in the message thread after a session change.	Fixed the message-thread query to always re-resolve the sender from the current request's authenticated user rather than a cached value, and added auto-refresh so both sides see new messages promptly.

11 Conclusion and Future Scope

Conclusion

The Local Civic Problem Reporting System successfully digitizes the process of reporting, tracking, and resolving local infrastructure problems. It replaces an undocumented paper-and-phone process with photo-and-location-backed reports, automatic category-and-ward routing to the responsible department, an immutable status-log audit trail, and a direct citizen-to-staff message thread. Role-based access control, bcrypt password hashing, stateless JWT authentication, and parameterized database access secure the system end to end, and the two-service Render deployment against a managed PostgreSQL database demonstrates a working, publicly reachable e-governance service that improves the transparency and accountability of local civic problem resolution.

Future Scope

Table 11.1 Future Scope

S.N.	Future Enhancement	Description
1	SMS / push notifications	Notify a citizen automatically when their report's status changes, for citizens without reliable app/email access.
2	Admin analytics dashboard	Turnout-style charts of report volume per category/ward and average time-to-resolution, to identify recurring problem areas.
3	Native mobile app	Wrap the existing REST API in a Flutter/React Native app with camera and GPS integration and offline drafting of a report.
4	Multi-language support	Add a Nepali-language interface alongside English, matching the bilingual approach seen in national e-governance portals.
5	Duplicate-report detection	Flag likely duplicate reports (same ward + category + nearby coordinates) for Staff at review time.

12 References

- [1] Wikipedia, “E-governance.” [Online]. Available: <https://en.wikipedia.org/wiki/E-governance>
- [2] Department of National ID and Civil Registration, Government of Nepal. [Online]. Available: <https://donidcr.gov.np>
- [3] Department of Passports, Government of Nepal. (2068 B.S.) [Online]. Available: <https://nepalpassport.gov.np>
- [4] Inland Revenue Department, Government of Nepal. [Online]. Available: <https://ird.gov.np>
- [5] FastAPI documentation. [Online]. Available: <https://fastapi.tiangolo.com>
- [6] SQLAlchemy documentation. [Online]. Available: <https://docs.sqlalchemy.org>
- [7] Render, “Render Docs.” [Online]. Available: <https://render.com/docs>
- [8] Neon, “Neon Serverless Postgres Docs.” [Online]. Available: <https://neon.tech/docs>
- [9] PyJWT / python-jose documentation — JSON Web Token implementation for Python. [Online]. Available: <https://python-jose.readthedocs.io>
- [10] Leaflet, “An open-source JavaScript library for mobile-friendly interactive maps.” [Online]. Available: <https://leafletjs.com>